Firebase aplicado a Scrapp: autenticación y persistencia en la nube para el control de mermas

Víctor Emmanuel Velasco Martínez (VM251307)

Fernando José Gómez Hernández (GH251230)

José Eduardo Aquino Medrano (AM252078)

William Eduardo Montano Aguilar (MA251192)

Grupo CTRL+CAFE

Dirección de Educación a Distancia, Universidad Don Bosco

DSM941 G01T: Desarrollo de Software para Móviles

Alexander Alberto Siguenza Campos

8 de noviembre de 2026

Proyecto de cátedra. Etapa 3, Avance 2: Foro de Discusión 2

Índice

Introducción 5
Firebase como plataforma de servicios en la nube 5
Qué es Firebase 5
Firebase Authentication 6
Cloud Firestore 6
Por qué Firebase y no un backend propio 7
Configuración de Firebase en el proyecto 8
Creación del proyecto y registro de la app 8
Dependencias 9
Huellas SHA-1 y cliente web de OAuth 10
Arquitectura general 10
Flujo de autenticación 12
Correo y contraseña 12
Inicio de sesión con Google mediante Credential Manager 14
Manejo de errores 15
Sesión persistente, navegación y cuentas desactivadas 16
Arquitectura de datos en Firestore 17
Modelo de colecciones 17
Decisiones de modelado 19
Asociación de los datos al usuario autenticado 20
Datos iniciales 21
Reglas de seguridad 22
Funciones auxiliares y reglas por colección 22
Matriz de permisos por rol 24
Las reglas no son filtros 24
Sincronización en tiempo real y modo sin conexión 25

Del repositorio síncrono al listener 25
Atomicidad del registro de merma 28
Listeners ligados a la sesión 29
Persistencia sin conexión 29
Módulos secundarios construidos sobre Firebase 30
Historial de mermas y de lotes 30
Panel de gerencia y configuración de motores 30
Gestión de usuarios y catálogo de productos 31
Evidencia fotográfica 32
Análisis reflexivo 33
Qué nos resolvió Firebase 33
Limitaciones encontradas 34
Qué haríamos distinto 35
Conclusiones 35
Referencias 37

Índice de figuras

Figura 1 *Proveedores de inicio de sesión habilitados en Firebase Console* 9

Figura 2 *Arquitectura de Scrapp con Firebase Authentication y Cloud Firestore* 11

Figura 3 *Secuencia de inicio de sesión con correo y contraseña* 13

Figura 4 *Secuencia de inicio de sesión con Google* 14

Figura 5 *Pantalla de inicio de sesión con el botón «Continuar con Google»* 15

Figura 6 *Modelo de datos de Scrapp en Cloud Firestore* 18

Figura 7 *Documento de la colección mermas en Firebase Console* 21

Figura 8 *Funciones auxiliares y regla de la colección lotes en firestore.rules* 23

Figura 9 *Recorrido de una merma desde el registro en un dispositivo hasta la recomposición en otro* 26

Figura 10 *Registro de una merma y descuento del lote en una sola escritura atómica* 28

Figura 11 *Panel de gerencia con gráficas alimentadas por Firestore* 31

Figura 12 *Registro de merma con la evidencia subiendo a Cloudinary* 33

Índice de tablas

Tabla 1 *Comparación entre un backend propio y Firebase para las necesidades de Scrapp* 7

Tabla 2 *Dependencias de Firebase e identidad declaradas en el proyecto* 9

Tabla 3 *Traducción de excepciones de Firebase a mensajes para el usuario* 16

Tabla 4 *Colecciones de Firestore y su propósito en Scrapp* 19

Tabla 5 *Permisos de cada rol por colección según firestore.rules* 24

Tabla 6 *Pantallas de Scrapp y los datos en tiempo real que observan* 27

Tabla 7 *Limitaciones de Firebase encontradas en Scrapp y cómo las abordamos* 34

Introducción

Scrapp es la aplicación móvil que el grupo CTRL+CAFE desarrolla para registrar y controlar las mermas de un restaurante: el producto que se pierde por caducidad, sobreproducción, mal almacenamiento o error de preparación. En la Etapa 2 la lógica del sistema funcionaba en consola con datos en memoria, y en el Avance 1 de la Etapa 3 la trasladamos a Android con Jetpack Compose. Esa versión ya autenticaba con Firebase por correo, pero los productos, lotes, ventas y mermas seguían viviendo en repositorios en memoria y desaparecían al cerrar la app.

Este documento explica cómo integramos Firebase Authentication y Cloud Firestore en Scrapp durante el Avance 2. Cada servicio aparece junto con la decisión técnica que tomamos en el proyecto y su motivo. Se describe la configuración del proyecto en Firebase, el flujo de autenticación por correo y por Google, el modelo de datos en Firestore, las reglas de seguridad por rol, la sincronización en tiempo real entre dispositivos y los módulos secundarios que se apoyan en estos servicios. Al final se incluye un análisis de lo que Firebase nos resolvió y de las limitaciones que encontramos.

El código citado corresponde al repositorio `scrapp-android` del grupo, paquete `com.ctrlcafe.scrapp`, en la rama `main` (versión `0.4-avance2`). Los diagramas son de elaboración propia a partir de ese código.

Firebase como plataforma de servicios en la nube

Qué es Firebase

Firebase es una plataforma de Google que ofrece servicios administrados para aplicaciones móviles y web: autenticación, bases de datos, almacenamiento de archivos, mensajería, analítica y alojamiento, entre otros. La idea es que el equipo consuma esas funciones

desde el cliente con los SDK oficiales, sin construir ni mantener un servidor propio (Moroney, 2017). Para Android, los servicios se agregan al proyecto con el complemento de Gradle `google-services` y un archivo de configuración generado en la consola (Firebase, s. f.-b).

Scrapp usa dos servicios de la plataforma, ambos dentro del plan gratuito Spark: Firebase Authentication para la identidad de los usuarios y Cloud Firestore como base de datos del negocio.

Firebase Authentication

Firebase Authentication administra el registro, el inicio de sesión y la sesión de los usuarios. Admite cuentas con correo y contraseña, proveedores federados como Google y otros métodos, y entrega a la app un usuario con un identificador único (`uid`) que se mantiene igual entre dispositivos y proveedores (Firebase, s. f.-g). En Scrapp, ese `uid` une la autenticación con los datos: es el id del documento de perfil en `usuarios/{uid}` y el valor que se guarda en `creadoPor` o `usuarioId` en cada registro del negocio.

El SDK también conserva la sesión en el dispositivo. Si el usuario cierra la app y vuelve a abrirla, `FirebaseAuth.currentUser` sigue disponible sin pedir la contraseña de nuevo, y con eso quedó resuelta la sesión persistente sin escribir código para guardar credenciales (Firebase, s. f.-c).

Cloud Firestore

Cloud Firestore es una base de datos NoSQL orientada a documentos. Los datos se guardan en documentos, que son conjuntos de pares campo-valor, y los documentos se agrupan en colecciones; un documento puede contener subcolecciones (Firebase, s. f.-e). No hay tablas ni

esquema fijo, y tampoco existe la operación `JOIN`: si una pantalla necesita datos de dos colecciones, la app debe leer ambas o duplicar los campos que necesita.

Hubo tres características de Firestore que pesaron en la decisión. Una es la escucha en tiempo real: un cliente puede registrar un *listener* sobre un documento o una consulta y recibe una instantánea nueva cada vez que los datos cambian (Firebase, s. f.-h). Otra es la persistencia sin conexión, que en Android viene activada por defecto: las lecturas se sirven desde una caché local y las escrituras se encolan hasta que vuelve la red (Firebase, s. f.-a). Y están las reglas de seguridad, que se evalúan en el servidor antes de cada lectura o escritura y permiten controlar el acceso sin un backend intermedio (Firebase, s. f.-l).

Por qué Firebase y no un backend propio

En la Etapa 1 consideramos dos caminos para la persistencia en la nube: construir una API propia (por ejemplo, con Laravel y MySQL, que algunos integrantes ya conocían) o usar Firebase. La Tabla 1 resume la comparación que nos llevó a elegir Firebase.

Tabla 1

Comparación entre un backend propio y Firebase para las necesidades de Scrapp

Necesidad de Scrapp	Backend propio (API + base relacional)	Firebase (Auth + Firestore)
Inicio de sesión con correo y Google	Implementar registro, *hash* de contraseñas, OAuth con Google y manejo de tokens	Proveedores listos; la app solo intercambia credenciales con el SDK
Sincronizar lotes y mermas entre cocina y gerencia	WebSockets o consultas periódicas; servidor siempre encendido	`addSnapshotListener` en el cliente, sin servidor que mantener
Funcionar con señal inestable en cocina	Caché local y cola de escrituras hechas a mano	Persistencia sin conexión activada por defecto en Android
Restringir lo que ve cada rol	Middleware de autorización en cada endpoint	Reglas de seguridad evaluadas en el servidor de Firebase
Costo para un proyecto académico	Alojamiento del servidor y de la base de datos	Plan Spark sin costo dentro de sus cuotas diarias
Consultas con relaciones	`JOIN` y agregaciones en SQL	Sin `JOIN`; obliga a desnormalizar o a combinar en el cliente

Nota. Elaboración propia. La última fila es la principal desventaja de Firebase para Scrapp y se retoma en el análisis reflexivo.

Esto tiene un costo: renunciamos a las consultas relacionales y quedamos atados a un proveedor. A cambio, ninguno de los cuatro tuvo que mantener un servidor, y con doce días para el avance eso pesó bastante.

Configuración de Firebase en el proyecto

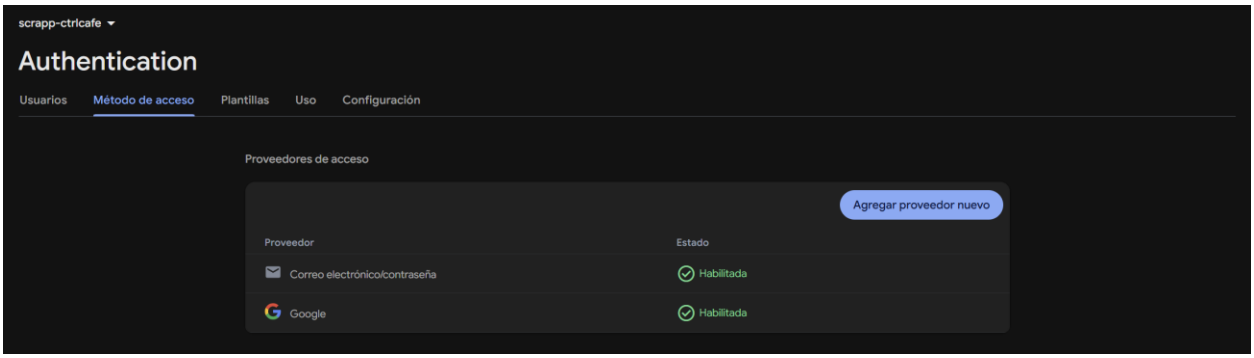
Creación del proyecto y registro de la app

El proyecto se creó en Firebase Console y en él se registró la app Android con el identificador de paquete `com.ctrlcafe.scrapp`. La consola generó el archivo `google-services.json`, que se colocó en `app/` y se versiona en el repositorio para que el docente pueda compilar sin pasos adicionales. Luego se habilitaron dos proveedores en Authentication (correo/contraseña y Google) y se creó la base de datos de Firestore en modo producción, es decir, con todo el acceso denegado hasta publicar nuestras reglas (Firebase, s. f.-b). La Figura 1 muestra los proveedores habilitados.

El complemento `com.google.gms.google-services` lee ese archivo durante la compilación y genera recursos como `default_web_client_id`, que más adelante usa el inicio con Google. Como un integrante que clona el repositorio sin el archivo no podría compilar, en `app/build.gradle.kts` el complemento se aplica solo si el archivo existe y, si falta, Gradle muestra una advertencia en lugar de fallar.

Figura 1

Proveedores de inicio de sesión habilitados en Firebase Console



Nota. Captura propia del proyecto de Firebase de Scrapp.

Dependencias

Las bibliotecas de Firebase se declaran a través de la *Bill of Materials* (BoM), que fija versiones compatibles entre sí; así no se indica la versión de cada módulo por separado (Firebase, s. f.-b). La Tabla 2 lista las dependencias relacionadas con Firebase y con el inicio de sesión con Google.

Tabla 2

Dependencias de Firebase e identidad declaradas en el proyecto

Dependencia	Versión	Uso en Scrapp
`com.google.firebase:firebase-bom`	34.2.0	Alinea las versiones de los módulos de Firebase
`firebase-auth`	BoM	Registro, inicio de sesión, sesión persistente y recuperación de contraseña
`firebase-firestore`	BoM	Repositorios, listeners en tiempo real y `WriteBatch`
`kotlinx-coroutines-play-services`	1.10.2	Función `await()` para esperar las tareas de Firebase dentro de corrutinas
`androidx.credentials:credentials` y `credentials-play-services-auth`	1.3.0	Credential Manager, que muestra el selector de cuentas de Google

Dependencia	Versión	Uso en Scrapp
`com.google.android.libraries.identity.googleid:googleid`	1.1.1	`GetGoogleIdOption` y `GoogleIdTokenCredential`
Complemento `com.google.gms.google-services`	4.4.3	Procesa `google-services.json` en la compilación

Nota. Elaboración propia a partir de `gradle/libs.versions.toml` y `app/build.gradle.kts`.

Huellas SHA-1 y cliente web de OAuth

El inicio con Google exige registrar en la consola la huella SHA-1 del certificado con que se firma la app (Firebase, s. f.-d). Cada integrante compila con su propio *keystore* de depuración, así que cada uno obtuvo su huella con `./gradlew signingReport` y el Integrante 2 las registró todas antes de descargar de nuevo `google-services.json`. Sin ese paso, Credential Manager devuelve un error en la máquina de quien no tenga su huella registrada.

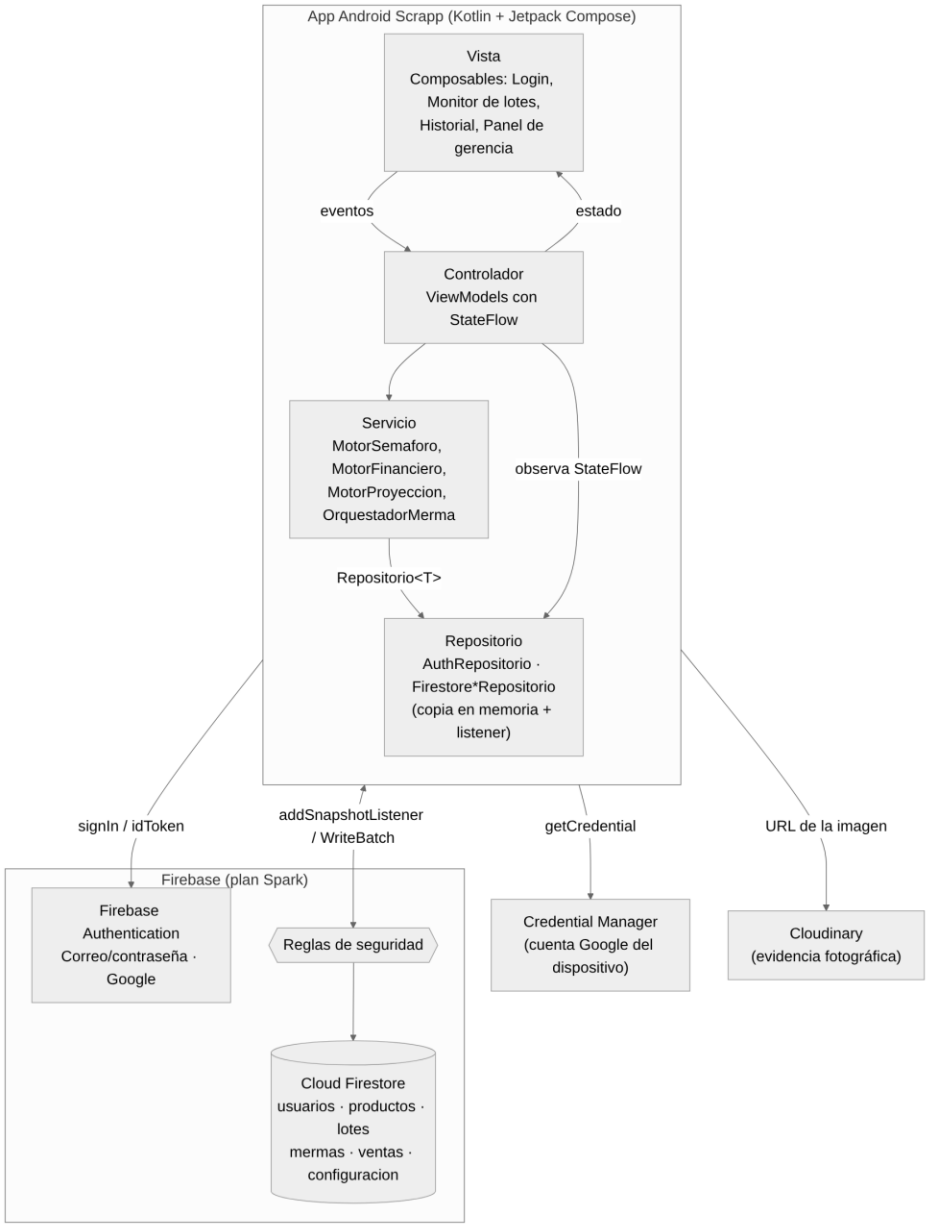
Además, la solicitud de credencial de Google debe incluir el identificador del cliente web de OAuth, no el de Android, porque el token que devuelve Google debe estar dirigido al servidor que lo valida, que en este caso es Firebase (Android Developers, s. f.-a). En Scrapp se toma del recurso generado `R.string.default_web_client_id`.

Arquitectura general

La Figura 2 muestra dónde entra Firebase en la arquitectura MVC que mantenemos desde la Etapa 1. Las pantallas en Compose no hablan con Firebase: envían eventos a los ViewModels y observan su estado. Solo la capa de repositorio conoce el SDK, de modo que los motores de cálculo de la Etapa 2 (`MotorSemaforo`, `MotorFinanciero`, `MotorProyeccion`) siguieron funcionando sin cambios sobre la interfaz `Repositorio<T>`.

Figura 2

Arquitectura de Scrapp con Firebase Authentication y Cloud Firestore



Nota. Las flechas indican quién llama a quién. Firestore solo se alcanza a través de las reglas de seguridad. Elaboración propia.

Flujo de autenticación

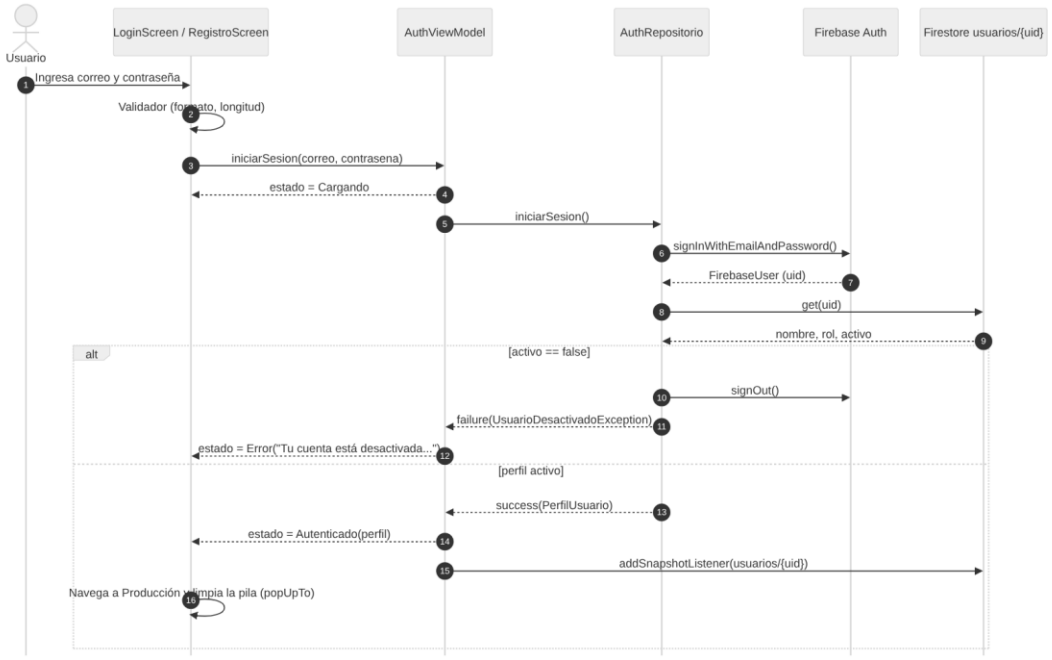
La autenticación está en `AuthRepositorio`, que encapsula Firebase Auth y la lectura del perfil en Firestore, y en `AuthViewModel`, que expone a las pantallas un estado sellado `AuthEstadoUi` con los valores `Inactivo`, `Cargando`, `Autenticado`, `Error`, `MensajeInfo` y `CuentaDesactivada`. Al ser un tipo cerrado, la pantalla de inicio de sesión no puede caer en un estado sin manejar: muestra el formulario, un indicador de carga, un mensaje o navega.

Correo y contraseña

Antes de llamar a Firebase, `Validador` revisa el formato del correo y la longitud de la contraseña, y la pantalla muestra el error bajo el campo correspondiente. Si los datos son válidos, `AuthRepositorio.iniciarSesion` llama a `signInWithEmailAndPassword` y, con el `uid` que devuelve, lee el documento `usuarios/{uid}` para conocer el rol y si la cuenta está activa (Firebase, s. f.-c). El proceso completo se muestra en la Figura 3.

Figura 3

Secuencia de inicio de sesión con correo y contraseña



Nota. Si el perfil está desactivado, la sesión de Firebase se cierra antes de devolver el error, para que no quede un usuario autenticado sin permisos. Elaboración propia.

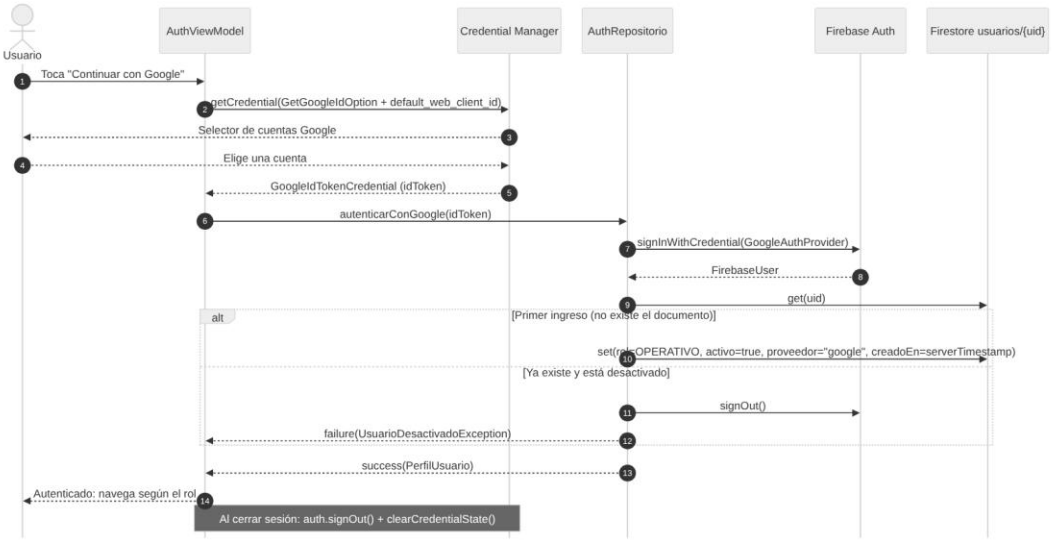
Crear una cuenta implica dos operaciones independientes: `createUserWithEmailAndPassword` en Authentication y la escritura del perfil en Firestore. Si la segunda falla (por ejemplo, por falta de red), quedaría una cuenta sin perfil que no podría entrar nunca, porque todas las pantallas dependen del rol. Para evitarlo, `registrar` intenta guardar el perfil y, si falla, elimina la cuenta recién creada con `usuario.delete()` antes de propagar el error. Todo usuario nuevo entra con rol `OPERATIVO`, `activo = true`, `proveedor = "password"` y `creadoEn` con la hora del servidor.

Inicio de sesión con Google mediante Credential Manager

Para Google usamos Credential Manager, la API que Android recomienda actualmente para el inicio de sesión con Google en lugar de la biblioteca anterior de Google Sign-In (Android Developers, s. f.-a). `AuthViewModel` construye una `GetGoogleIdOption` con el identificador del cliente web, sin filtrar por cuentas ya autorizadas y sin selección automática, para que el usuario siempre vea el selector. Con el token de identidad que devuelve Credential Manager se crea una credencial de `GoogleAuthProvider` y se inicia sesión en Firebase con `signInWithCredential` (Firebase, s. f.-d). La Figura 4 muestra la secuencia y la Figura 5, la pantalla resultante.

Figura 4

Secuencia de inicio de sesión con Google



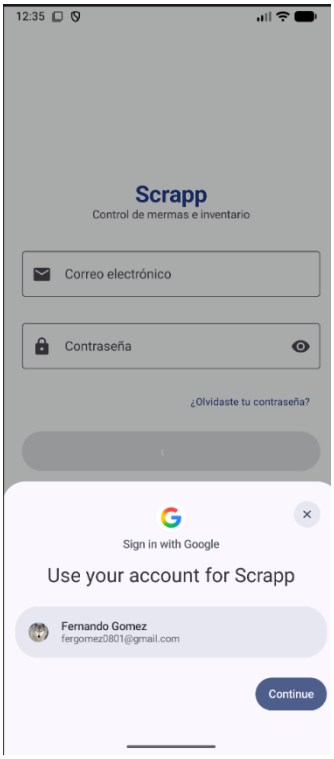
Nota. En el primer ingreso se crea el perfil con el mismo rol inicial que en el registro por correo. Elaboración propia.

Como Google no pasa por nuestra pantalla de registro, el perfil se crea en el primer ingreso: si `usuarios/{uid}` no existe, `autenticarConGoogle` lo escribe con el nombre y el

correo de la cuenta de Google, rol `OPERATIVO` y `proveedor = "google"`. Si existe pero está desactivado, se cierra la sesión y se informa al usuario. Si la persona cancela el selector de cuentas, Credential Manager lanza `GetCredentialCancellationException` y el ViewModel vuelve a `Inactivo` sin mostrar ningún error, porque cancelar no es una falla.

Figura 5

Pantalla de inicio de sesión con el botón «Continuar con Google»



Nota. Captura propia de la app Scrapp.

Manejo de errores

Las excepciones de Firebase traen mensajes técnicos en inglés que no le sirven al personal de cocina. `AuthRepositorio` envuelve cada operación en una función `ejecutar` que devuelve `Result<T>` y traduce la excepción original a una excepción propia de Scrapp con un

mensaje en español, como se resume en la Tabla 3. El detalle técnico se registra con `Logger` para depurar, pero no se muestra en pantalla.

Tabla 3

Traducción de excepciones de Firebase a mensajes para el usuario

Excepción de Firebase	Mensaje que ve el usuario
`FirebaseAuthInvalidCredentialsException`	Correo o contraseña incorrectos.
`FirebaseAuthInvalidUserException` (`ERROR_USER_DISABLED`)	Tu cuenta está desactivada. Contactá al administrador.
`FirebaseAuthUserCollisionException`	Ya existe una cuenta con ese correo.
`FirebaseAuthWeakPasswordException`	La contraseña debe tener al menos 6 caracteres.
`FirebaseNetworkException` o Firestore `UNAVAILABLE`	No hay conexión a internet. Revisá tu red e intentá de nuevo.
Perfil inexistente en `usuarios/{uid}`	No se encontró el perfil del usuario. Contactá al administrador.

Nota. Elaboración propia a partir de `AuthRepositorio.traducir()` y `util/Excepciones.kt`.

La colisión de correos aparece, por ejemplo, cuando alguien intenta registrarse con correo y contraseña usando una dirección con la que ya entró por Google. En lugar del error crudo de Firebase, el usuario ve que la cuenta ya existe y puede usar el botón de Google.

Sesión persistente, navegación y cuentas desactivadas

Al abrir la app, el grafo de navegación arranca en una ruta `Splash`. `AuthViewModel.verificarSesionActiva` consulta `currentUser` y, si hay sesión, lee el perfil y navega directamente al módulo que corresponde al rol; si no, muestra el inicio de sesión. Al pasar del login a la app principal se usa `popUpTo` con `inclusive = true`, de modo que el botón atrás no regresa al login, y al cerrar sesión se limpia toda la pila para que tampoco se pueda volver a una pantalla protegida.

Para cuentas desactivadas por el administrador mientras el usuario tiene la sesión abierta, `AuthRepositorio.observarEstadoUsuarioActual` abre un *listener* sobre el propio documento del usuario y lo expone como `Flow<Boolean>` con `callbackFlow`. Cuando `activo` pasa a `false`, el ViewModel cierra la sesión y cambia a `CuentaDesactivada`, y la navegación lleva al login con un aviso. Pasa en segundos y sin reiniciar la app. Las reglas de Firestore bloquean además cualquier lectura de un usuario inactivo, así que la protección no depende solo del cliente.

Al cerrar sesión se llama a `auth.signOut()` y a `CredentialManager.clearCredentialState()`. Sin lo segundo, el siguiente inicio con Google en el mismo teléfono reutilizaría la cuenta anterior, algo probable en una cocina donde varios empleados usan la misma tableta (Android Developers, s. f.-a). Por último, `RecuperarContrasenaScreen` valida el correo y llama a `sendPasswordResetEmail`; la confirmación se muestra con el estado `MensajeInfo` (Firebase, s. f.-c).

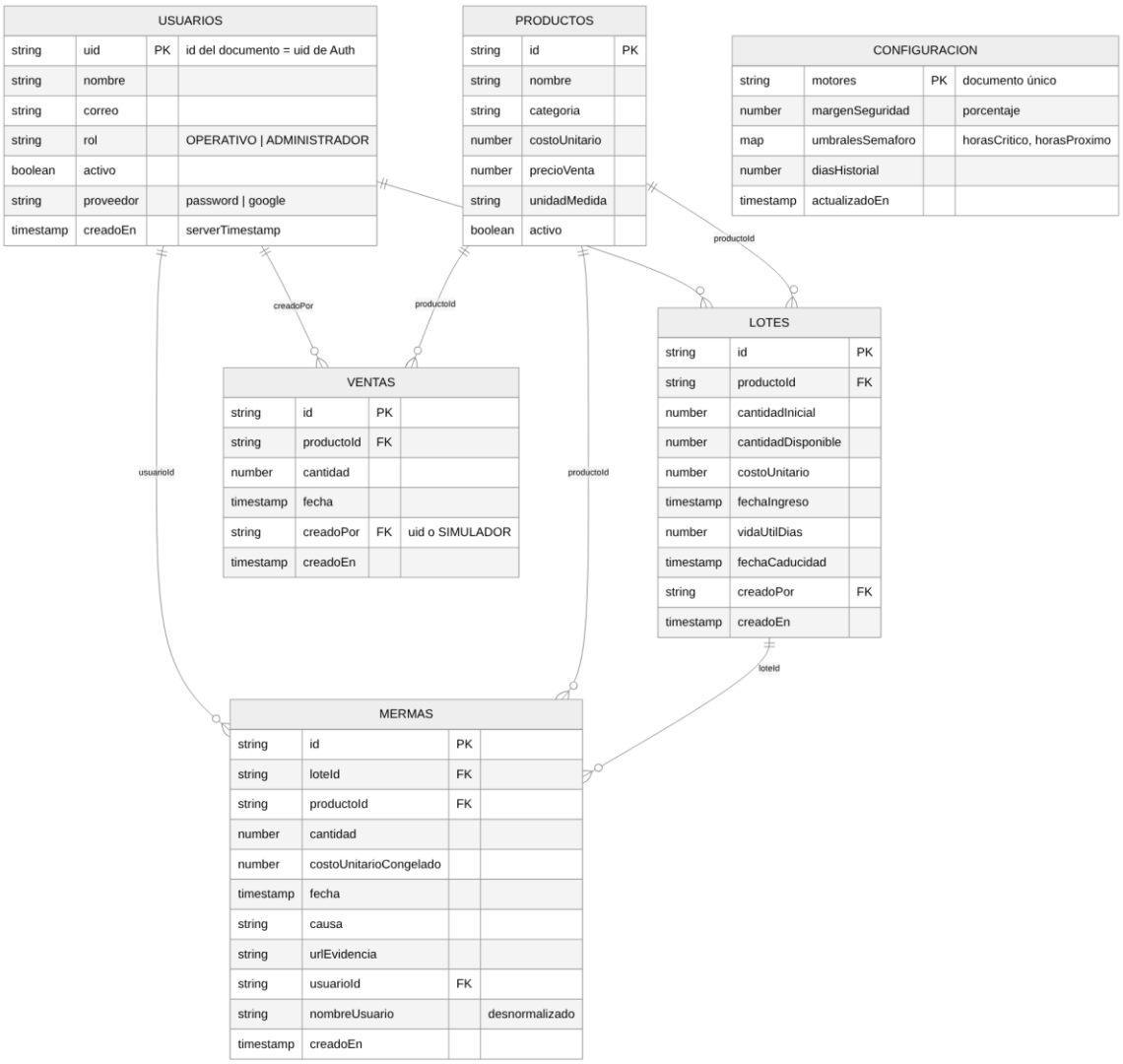
Arquitectura de datos en Firestore

Modelo de colecciones

El modelo se acordó en el grupo antes de escribir cualquier repositorio, para que nadie creara colecciones por su cuenta. Son seis colecciones en la raíz de la base de datos. La Figura 6 muestra sus campos y las referencias entre ellas, y la Tabla 4 resume para qué sirve cada una.

Figura 6

Modelo de datos de Scrapp en Cloud Firestore



Nota. Firestore no tiene llaves foráneas; las relaciones son campos de texto que guardan el id del documento referido (marcados como FK). Elaboración propia.

Tabla 4

Colecciones de Firestore y su propósito en Scrapp

Colección	Contenido	Quién escribe
`usuarios`	Perfil de cada cuenta: nombre, correo, rol, estado y proveedor. El id del documento es el `uid` de Authentication	El propio usuario al registrarse; el administrador cambia rol y estado
`productos`	Catálogo con costo, precio de venta y unidad de medida	Administrador
`lotes`	Ingresos de producto con cantidad disponible y fecha de caducidad	Operativo (crea y descuenta) y administrador
`mermas`	Cada pérdida registrada, con causa, costo y evidencia	Operativo y administrador
`ventas`	Ventas que usa `MotorProyeccion`; incluye ventas simuladas con id `SIM-*` para mantener la recomendación al día	Administrador; cualquier usuario activo crea solo ventas simuladas
`configuracion`	Documento `motores` con el margen de seguridad (en porcentaje) y los umbrales del semáforo en horas (`horasCritico`, `horasProximo`)	Administrador

Nota. Elaboración propia.

Decisiones de modelado

Colecciones en la raíz en lugar de subcolecciones. Se pudo guardar las mermas como subcolección de cada lote (`lotes/{id}/mermas`), pero el historial y el panel de gerencia necesitan consultar todas las mermas a la vez, filtradas por fecha, producto o usuario. Con colecciones en la raíz y referencias por id, esas consultas son directas (Firebase, s. f.-e).

Desnormalización del nombre de usuario. Cada merma guarda el nombre de quien la registró, además de su `usuarioId`. Como Firestore no tiene `JOIN`, mostrar el nombre en cada fila del historial obligaría a leer `usuarios` por cada merma, lo que suma lecturas y, además, choca con las reglas, porque un operativo no puede leer el perfil de otros. Duplicar el nombre es más barato y el dato casi nunca cambia.

Costo congelado. La merma guarda `costoUnitarioCongelado`, el costo del producto en el momento del registro. Si el administrador actualiza el costo en el catálogo, las pérdidas del

mes pasado no deben recalcularse con el precio nuevo. Es la misma regla de negocio de la Etapa 2, ahora reflejada en el documento.

El semáforo no se guarda. El estado verde, amarillo o rojo de un lote depende de la fecha de hoy, así que guardarlo obligaría a actualizar documentos todos los días. En su lugar se guarda `fechaCaducidad` y `MotorSemaforo` calcula el estado al leer, con los umbrales de `configuracion/motores`.

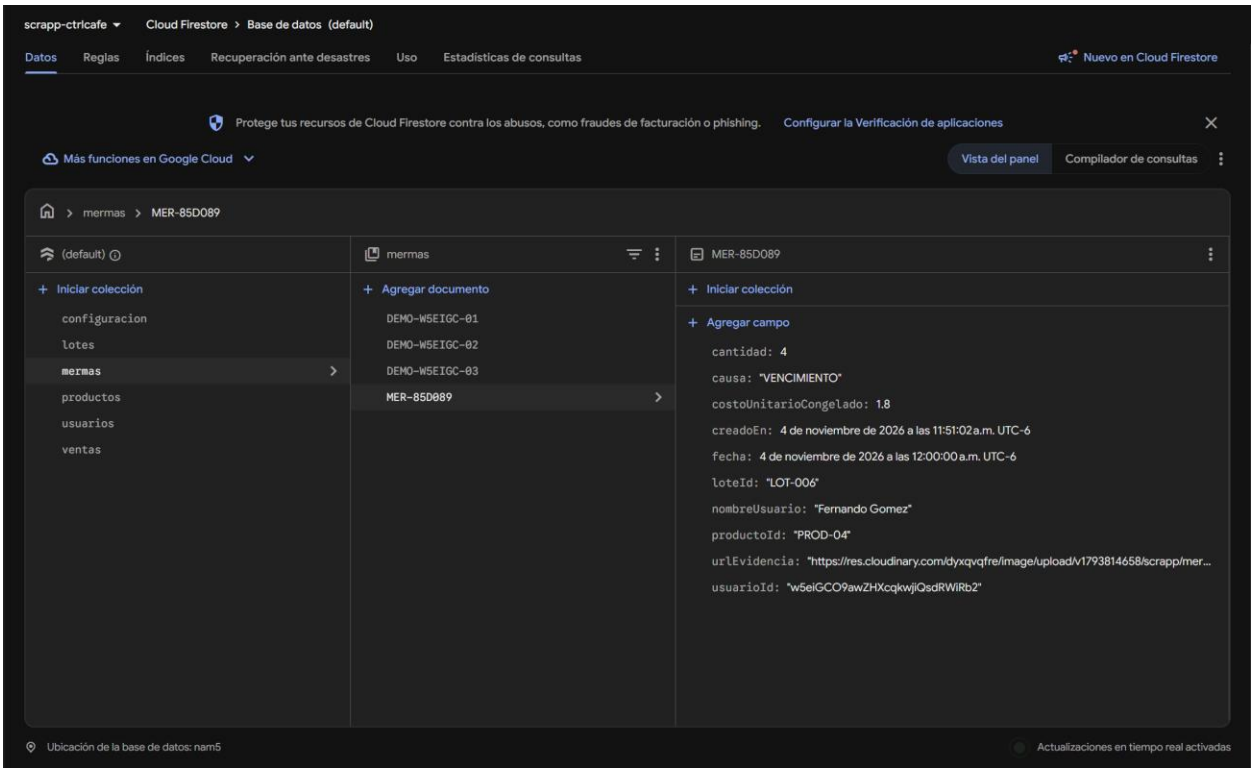
Fechas y tipos. Las fechas se guardan como `Timestamp` de Firestore y la conversión a `LocalDate` está concentrada en `Mapeadores.kt`, para no repetirla en cada repositorio. Los campos de auditoría (`creadoEn`, `actualizadoEn`) usan `FieldValue.serverTimestamp()` y no la hora del teléfono, que puede estar mal configurada. Las enumeraciones como `causa` o `rol` se guardan con su nombre en texto, y al leer se convierten con un valor por defecto si llega algo desconocido, para que un dato mal escrito en la consola no cierre la app.

Asociación de los datos al usuario autenticado

Cada registro de negocio identifica a su autor con el `uid` de Firebase Authentication: `lotes.creadoPor`, `ventas.creadoPor` y `mermas.usuarioId`. La Figura 7 muestra un documento real de la colección `mermas`. El valor no lo escribe el usuario, sino el ViewModel a partir de la sesión activa, y las reglas de seguridad rechazan cualquier escritura en la que no coincida con `request.auth.uid`, así que el campo no se puede falsear desde la app.

Figura 7

Documento de la colección mermas en Firebase Console



Nota. Captura propia del proyecto de Firebase de Scrapp.

Datos iniciales

Para que el docente encuentre datos al clonar el proyecto y entrar por primera vez, `SembradorDatos` consulta si la colección `productos` tiene al menos un documento. Si está vacía, carga los datos de ejemplo de `DatosIniciales` (los mismos de la Etapa 2) y los sube a Firestore junto con la configuración inicial de los motores. La siembra se dispara solo cuando ya hay una sesión, porque antes de iniciar sesión las reglas rechazan la consulta.

Con el uso aparecieron dos huecos en esos datos. Las mermas de ejemplo pertenecen a usuarios ficticios y un operativo solo ve las suyas, así que quien entraba por primera vez con Google encontraba el historial vacío. Para eso,

`SembradorDatos.sembrarMermasDemoSiNoTiene` le crea seis mermas de prueba a su nombre (ids `DEMO-…`) con fechas del mes en curso; no descuentan existencias de los lotes. El otro hueco era la recomendación de producción: `MotorProyeccion` promedia las ventas del mismo día de la semana de las cuatro semanas anteriores, y las ventas sembradas una sola vez quedan fuera de esa ventana con el paso de los días. Al iniciar sesión, como máximo una vez al día por dispositivo, la app completa las ventas de los últimos 35 días de cada producto activo que no tengan registro. Esas ventas son simuladas, llevan ids `SIM-{producto}-{yyyyMMdd}` y `creadoPor = "SIMULADOR"`, se escriben en lotes de hasta 500 operaciones y nunca reemplazan una venta real.

Reglas de seguridad

Las reglas fueron la parte de Firebase que más tiempo nos llevó. Como la app se comunica directamente con la base de datos, cualquiera que obtenga la configuración del proyecto podría intentar leer o escribir sin pasar por nuestras pantallas. Las validaciones de la interfaz mejoran la experiencia, pero la seguridad real está en `firestore.rules`, que se evalúa en el servidor (Firebase, s. f.-l).

Funciones auxiliares y reglas por colección

Las reglas comienzan con tres funciones. `autenticado()` exige una sesión; `esActivo()` exige además que exista el perfil y que `activo` sea verdadero, y `esAdmin()` agrega que el rol sea `ADMINISTRADOR`. Todas las demás reglas se escriben a partir de ellas, lo que cumple el requisito de que un usuario desactivado no pueda leer ni escribir nada. La Figura 8 muestra estas funciones y la regla de la colección `lotes`, que es la más elaborada.

Figura 8

Funciones auxiliares y regla de la colección lotes en firestore.rules

```
function perfil() {
  return get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data;
}
function esActivo() {
  return autenticado()
    && exists(/databases/$(database)/documents/usuarios/$(request.auth.uid))
    && perfil().activo == true;
}
function esAdmin() { return esActivo() && perfil().rol == 'ADMINISTRADOR'; }

match /lotes/{id} {
  allow read: if esActivo();
  // El operativo crea lotes solo a su nombre
  allow create: if esAdmin()
    || (esActivo() && request.resource.data.creadoPor == request.auth.uid);
  // El operativo solo puede bajar cantidadDisponible (descuento por merma)
  allow update: if esAdmin() || (
    esActivo()
    && request.resource.data.diff(resource.data).affectedKeys()
         .hasOnly(['cantidadDisponible'])
    && request.resource.data.cantidadDisponible < resource.data.cantidadDisponible
  );
  allow delete: if esAdmin();
}
```

Nota. Fragmento de `firestore.rules` del repositorio `scrapp-android`.

La regla de actualización de lotes resolvió un conflicto que detectamos al planificar el avance. La primera versión solo dejaba escribir lotes al administrador, pero registrar una merma implica descontar la cantidad disponible del lote, y quien registra mermas es el personal operativo. Abrir toda la escritura de lotes al operativo habría permitido, por ejemplo, cambiar la fecha de caducidad de un lote vencido. Con `diff().affectedKeys().hasOnly()` el operativo puede modificar un único campo, y solo para disminuirlo.

En `usuarios`, un usuario solo puede crear su propio documento y solo con rol `OPERATIVO` y `activo = true`, así que nadie puede registrarse directamente como administrador. Después, el usuario solo puede cambiar su nombre; el rol y el estado los cambia el administrador.

La colección `ventas` tiene una excepción a la regla de solo administrador, que necesitan las ventas simuladas descritas en la sección de datos iniciales. Cualquier usuario activo puede crear un documento de ventas, pero solo si su id empieza con `SIM-`, `creadoPor` es `SIMULADOR` y la cantidad es un número mayor que cero. Editar o borrar ventas, y crear ventas reales, sigue reservado al administrador.

Matriz de permisos por rol

La Tabla 5 resume lo que puede hacer cada rol en cada colección una vez aplicadas todas las reglas.

Tabla 5

Permisos de cada rol por colección según firestore.rules

Colección	Operativo (activo)	Administrador
`usuarios`	Lee y crea solo su documento (rol `OPERATIVO`); edita solo `nombre`	Lee todos; cambia rol y estado; elimina
`productos`	Solo lectura	Lectura y escritura
`lotes`	Lee; crea con `creadoPor` propio; solo disminuye `cantidadDisponible`	Lectura y escritura completas
`mermas`	Lee solo las suyas; crea con `usuarioId` propio; no edita ni borra	Lee todas; edita y elimina
`ventas`	Lee; crea solo ventas simuladas (`SIM-*`, `creadoPor = SIMULADOR`)	Lectura y escritura
`configuracion`	Solo lectura	Lectura y escritura

Nota. Un usuario con `activo == false` no tiene ningún permiso, sin importar su rol. Elaboración propia.

Las reglas no son filtros

Al probar las mermas descubrimos que las reglas de Firestore no filtran resultados. Si una consulta podría devolver algún documento que el usuario no tiene permiso de leer, Firestore rechaza la consulta completa en lugar de devolver solo los permitidos (Firebase, s. f.-i). Un

listener sobre toda la colección `mermas` abierto con una cuenta operativa falla con `PERMISSION_DENIED`, aunque ese usuario tenga mermas propias.

La solución fue que la consulta dependa del rol: al iniciar sesión, el repositorio lee el perfil y, si el usuario es administrador, escucha toda la colección; si es operativo, escucha `whereEqualTo("usuarioId", uid)`. Esa consulta coincide exactamente con lo que la regla permite, y Firestore la acepta. Si el perfil todavía no existe, como ocurre justo después del registro, se asume el rol con menos permisos.

Sincronización en tiempo real y modo sin conexión

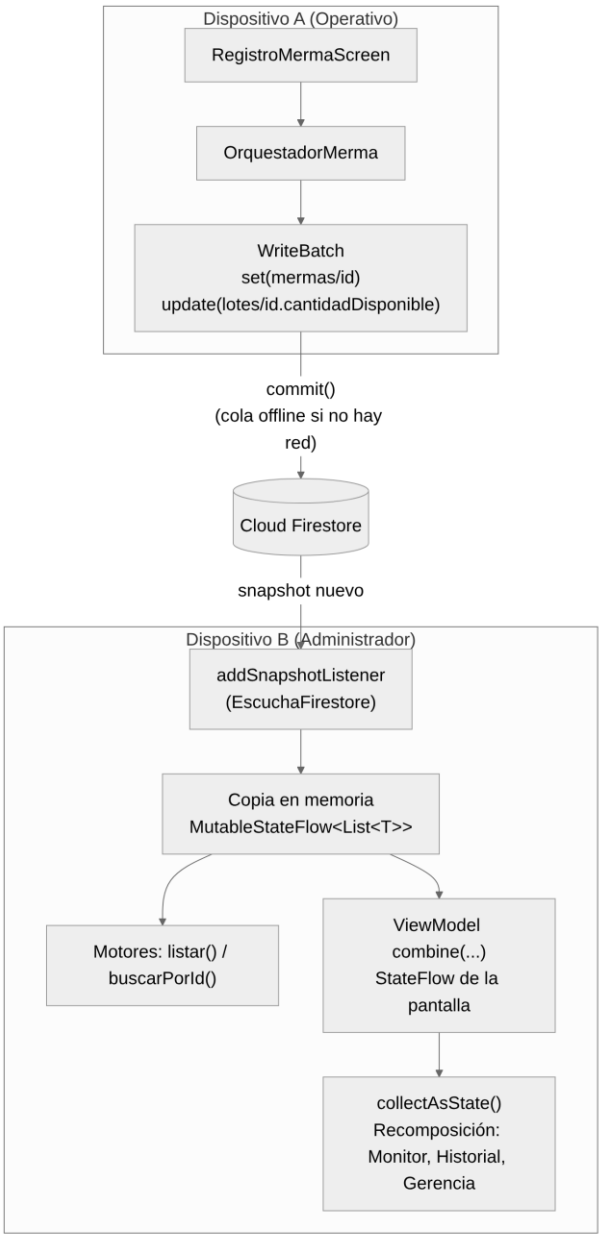
Del repositorio síncrono al listener

Lo más difícil del avance fue conectar Firestore, que es asíncrono, con los motores de la Etapa 2, que llaman a `listar()` y `buscarPorId()` de forma síncrona sobre `Repositorio<T>`. Reescribir los motores con corrutinas habría tocado todo el código probado. Acordamos que cada repositorio de Firestore mantenga una copia en memoria de su colección, alimentada por un *listener* (Firebase, s. f.-h).

Esa copia vive en un `MutableStateFlow<List<T>>`. Los motores leen su valor actual como antes, y los ViewModels se suscriben al mismo flujo y lo combinan con otros mediante `combine` para producir el estado de cada pantalla (Android Developers, s. f.-b). Cuando llega una instantánea nueva, el flujo emite, el ViewModel recalcula y Compose recompone la pantalla. La Figura 9 muestra el recorrido de una merma registrada en un dispositivo hasta la pantalla de otro.

Figura 9

Recorrido de una merma desde el registro en un dispositivo hasta la recomposición en otro



Nota. Las escrituras van a Firestore; el listener es el único que actualiza la copia en memoria de los demás dispositivos. Elaboración propia.

En el video de demostración esto se ve con dos emuladores abiertos: una merma registrada con la cuenta operativa aparece en el monitor de lotes, el historial y el panel de

gerencia de la cuenta administradora sin recargar. Lo mismo ocurre con `configuracion/motores`: `ConfiguracionRepositorio` aplica cada cambio del documento sobre la instancia de `ConfiguracionMotores` que usan los motores, y `ProduccionViewModel` combina ese flujo con los de lotes y productos. Si el administrador cambia el margen de seguridad o los umbrales del semáforo, la proyección de producción y los colores del monitor se recalculan en todos los dispositivos. La Tabla 6 resume qué observa cada pantalla.

Tabla 6

Pantallas de Scrapp y los datos en tiempo real que observan

Pantalla	Flujos que combina	Qué se actualiza solo
Dashboard de producción	`lotesFlow`, `productosFlow`, `ventasFlow`, `mermasFlow`, `configFlow`	Cantidades sugeridas de producción
Monitor de lotes	`lotesFlow`, `productosFlow`, `configFlow`	Cantidad disponible y color del semáforo
Historial de mermas	`mermasFlow`, `productosFlow`, `lotesFlow`, nombres de usuario	Nuevas filas y pérdida en dinero
Panel de gerencia	`mermasFlow`, `lotesFlow`, `productosFlow`	Pérdida del día y del mes, índice de merma y gráficas
Gestión de usuarios	Listener sobre `usuarios`	Rol y estado de cada cuenta
Configuración de motores	`configFlow`	Margen de seguridad y umbrales vigentes

Nota. Elaboración propia a partir de los ViewModels del paquete `controlador`.

El *listener* tarda un ciclo en reflejar lo que escribe el mismo dispositivo, pero `OrquestadorMerma` busca la merma recién creada en el mismo instante para recalcular indicadores. Por eso `FirestoreMermaRepositorio` agrega la merma a la copia local al escribir (`agregarACopiaLocal`), y el *listener* la reemplaza cuando llega la versión del servidor.

Atomicidad del registro de merma

Registrar una merma son dos escrituras: el documento nuevo en `mermas` y la cantidad actualizada del lote. Si solo se aplicara una, el inventario quedaría inconsistente. Para evitarlo, el repositorio implementa la interfaz `RegistroMermaAtomico` con un `WriteBatch`, que aplica todas sus operaciones o ninguna (Firebase, s. f.-j), como se ve en la Figura 10. La interfaz vive en `servicio/`, así que `OrquestadorMerma` no depende de Firestore.

Figura 10

Registro de una merma y descuento del lote en una sola escritura atómica

```
override fun registrarMermaYDescontarLote(merma: Merma, nuevaCantidad: Double): Merma {
    val batch = db.batch()
    val refMerma = if (merma.id.isBlank()) coleccion.document()
                   else coleccion.document(merma.id)
    val mermaFinal = merma.copy(id = refMerma.id)
    batch.set(refMerma, mapearDatos(mermaFinal))      // incluye serverTimestamp()

    val refLote = db.collection("lotes").document(merma.loteId)
    batch.update(refLote, "cantidadDisponible", nuevaCantidad)

    batch.commit()            // se encola si no hay red
    agregarACopiaLocal(mermaFinal)
    return mermaFinal
}
```

Nota. Fragmento simplificado de `FirestoreMermaRepositorio.kt`.

Elegimos `WriteBatch` y no una transacción porque las transacciones fallan cuando el cliente no tiene conexión (Firebase, s. f.-j), y en la cocina la señal se pierde con frecuencia. El lote ya está en memoria y el cálculo de la cantidad nueva lo hace `OrquestadorMerma`, así que no necesitamos la lectura dentro de la operación que da una transacción. La contrapartida se discute en el análisis reflexivo.

Listeners ligados a la sesión

`ContenedorDatos` crea los repositorios al iniciar la app, antes del login, porque `AuthViewModel` los necesita. Un *listener* abierto sin sesión falla con `PERMISSION_DENIED` y Firestore lo cierra de forma definitiva. Para resolverlo creamos `SesionFirestore`, que escucha los cambios de usuario de Firebase Auth y le avisa a cada repositorio para que abra sus *listeners* al iniciar sesión, los cierre al salir y los reabra si cambia la cuenta.

También encontramos una condición de carrera en el registro: la sesión existe un instante antes de que se escriba `usuarios/{uid}`, y en ese intervalo las reglas niegan la lectura porque `esActivo()` no encuentra el perfil. `EscuchaFirestore` reintenta la conexión hasta cinco veces, con una espera creciente, cuando el error es `PERMISSION_DENIED`, e ignora las respuestas de conexiones viejas mediante un contador de generación.

Persistencia sin conexión

No tuvimos que configurar la persistencia: en Android viene activada por defecto (Firebase, s. f.-a). Con la red apagada, la app sigue mostrando los datos de la caché y las mermas registradas se encolan; al volver la conexión se envían y los demás dispositivos las reciben. Para que el usuario sepa en qué estado está, `IndicadorSinConexion` escucha la red con un `NetworkCallback` y muestra una franja de aviso sobre la barra inferior mientras no hay internet; solo informa y no bloquea nada, porque Firestore sigue trabajando con su caché. Las operaciones que necesitan confirmación del servidor, como el ingreso de un lote con `crearConfirmado` o el guardado de la configuración de motores, se esperan con un tiempo límite para que la pantalla muestre un mensaje en lugar de quedarse cargando indefinidamente.

Módulos secundarios construidos sobre Firebase

Historial de mermas y de lotes

`HistorialMermasScreen` cumple el requerimiento de visualizar los registros almacenados. Muestra las mermas en tiempo real ordenadas por fecha, con producto, cantidad, causa, pérdida en dinero, quién la registró y una miniatura de la evidencia, y permite filtrar por rango de fechas, producto y causa. El operativo ve solo sus mermas y el administrador ve todas, de acuerdo con la consulta por rol descrita en las reglas. El orden y los filtros se aplican en el cliente sobre el flujo ya recibido, lo que evita crear índices compuestos en Firestore para cada combinación de filtros.

`HistorialLotesScreen` muestra los lotes que ya salieron del monitor, ya sea porque se agotaron o porque vencieron, con su fecha de cierre, las unidades registradas como merma y la pérdida pendiente de los lotes vencidos que todavía tienen existencias.

Panel de gerencia y configuración de motores

El panel de gerencia, solo para el administrador, muestra la pérdida del día, la del mes y el índice de merma, más dos gráficas hechas con la biblioteca Vico: la pérdida diaria del mes y los cinco productos con más desperdicio (Figura 11). Todo se calcula con `MotorFinanciero` sobre los flujos de Firestore, así que una merma registrada en cocina mueve las tarjetas y las gráficas en el momento. Desde el panel se abre `ConfiguracionMotoresScreen`, donde el administrador edita el margen de seguridad y los umbrales del semáforo en horas. Al guardar, los valores se aplican en memoria de inmediato y se escriben en `configuracion/motores` con `SetOptions.merge()`, para no borrar otros campos del documento; si el servidor no confirma en cinco segundos, la pantalla avisa que el cambio quedó en cola.

Figura 11

Panel de gerencia con gráficas alimentadas por Firestore



Nota. Captura propia de la app Scrapp.

Gestión de usuarios y catálogo de productos

`GestionUsuariosScreen` lista los usuarios en tiempo real y le permite al administrador cambiar el rol y activar o desactivar cuentas, siempre con un diálogo de confirmación. El administrador no puede desactivarse a sí mismo, para que el sistema nunca quede sin administrador. Al desactivar una cuenta, el *listener* de esa persona detecta el cambio y su sesión se cierra, como se explicó en el flujo de autenticación.

`CatalogoProductosScreen` permite crear, editar y desactivar productos. Los productos no se eliminan, porque hay lotes y mermas que los referencian por id y, sin `JOIN` ni integridad referencial en Firestore, un producto borrado dejaría filas del historial sin nombre.

Evidencia fotográfica

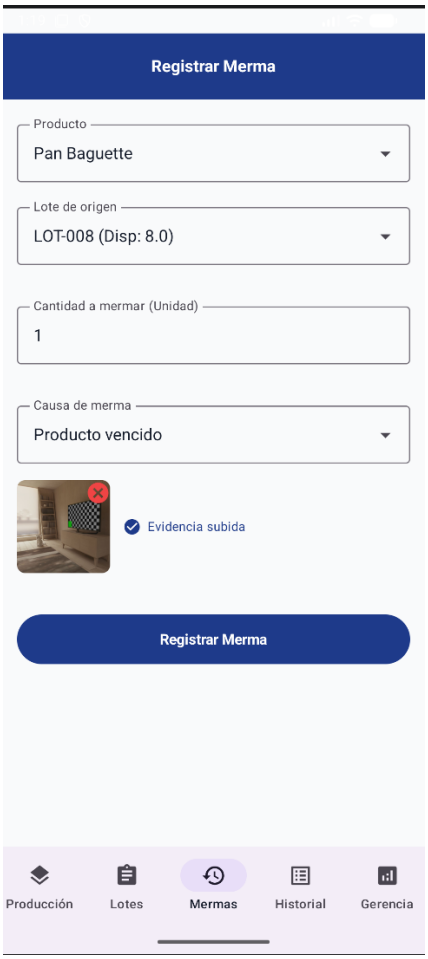
La foto de evidencia se toma con CameraX desde `CamaraEvidenciaScreen` o se elige de la galería, y en la merma se guarda la URL de la imagen en el campo `urlEvidencia`. Para almacenar la imagen evaluamos Cloud Storage for Firebase, pero desde septiembre de 2024 los buckets nuevos de Storage requieren el plan de pago Blaze, incluso para el uso sin costo (Firebase, s. f.-f). Como el proyecto debe funcionar sin registrar una tarjeta, elegimos Cloudinary con un *upload preset* sin firma, que ya habíamos propuesto en la Etapa 1. Firestore no guarda la imagen, solo su URL.

`EvidenciaRepositorio` prepara la foto antes de subirla: la reduce a 1600 px en su lado mayor, respeta la orientación EXIF y la recomprime en JPEG con calidad 80, de modo que una foto de varios MB queda en unos cientos de KB. Luego la envía por `POST` *multipart* a la API de subida de Cloudinary con el *preset* sin firma, en la carpeta `scrapp/mermas`, y emite el progreso como un `Flow<EstadoSubida>` que la pantalla muestra en una barra. De la respuesta se toma `secure_url`, que es lo que llega a Firestore. Mientras la foto no termine de subir, o si la subida falló, el botón de registrar queda deshabilitado y aparece la opción de reintentar, para que ninguna merma quede con una evidencia rota.

La app no lleva el *API secret* de Cloudinary. El *cloud name* y el nombre del *preset* se leen en la compilación desde `local.properties`, `gradle.properties` o variables de entorno y llegan al código como campos de `BuildConfig`. Si faltan, la app compila igual y avisa que Cloudinary no está configurado al adjuntar una foto.

Figura 12

Registro de merma con la evidencia subiendo a Cloudinary



Nota. Captura propia de la app Scrapp.

Análisis reflexivo

Qué nos resolvió Firebase

En doce días pasamos de datos en memoria a una app con cuentas reales, inicio con Google, datos compartidos entre dispositivos y funcionamiento sin conexión, sin escribir una línea de código de servidor. La sesión persistente, la cola de escrituras sin red y la notificación de cambios, que en un backend propio habrían sido trabajo aparte, ya venían en el SDK.

Lo que más nos ayudó fue una decisión de la Etapa 1: dejar la lógica detrás de `Repositorio<T>`. Por eso el cambio de memoria a Firestore se concentró en un paquete (`repositorio/firestore/`) y en `ContenedorDatos`, y los motores de cálculo no se tocaron.

También aprendimos que en Firebase la seguridad se piensa al revés. En un backend propio, el servidor decide qué devolver; en Firestore, el cliente pide y las reglas aceptan o rechazan. Esto nos obligó a pensar las consultas desde el permiso, como en el caso de las mermas por rol, y a tratar las reglas como parte del código.

Limitaciones encontradas

La Tabla 7 resume las limitaciones que encontramos durante el avance y cómo respondimos a cada una.

Tabla 7

Limitaciones de Firebase encontradas en Scrapp y cómo las abordamos

Limitación	Efecto en Scrapp	Respuesta del equipo
Sin `JOIN` entre colecciones	El historial necesitaba el nombre de quien registró cada merma	Desnormalizar `nombreUsuario` y combinar flujos en el ViewModel
Las reglas no filtran consultas	El *listener* de mermas fallaba completo para el operativo	Consulta por rol con `whereEqualTo("usuarioId", uid)`
Cloud Storage requiere plan Blaze	No se pudo alojar la evidencia en Firebase sin tarjeta	Cloudinary con *upload preset* sin firma; en Firestore solo la URL
`get()` en reglas se cobra como lectura	Cada operación evalúa el perfil del usuario y consume cuota	Aceptable en el plan Spark (50 000 lecturas diarias); se vigila en la consola
Índices compuestos para consultas con filtro y orden	Cada combinación de filtros del historial pediría un índice	Ordenar y filtrar en el cliente sobre el flujo recibido
Huella SHA-1 por máquina	Google no funciona en la máquina de quien no está registrado	Registrar las huellas de los cuatro integrantes antes de programar

Nota. Las cuotas gratuitas y el cobro de `get()` en reglas se basan en Firebase (s. f.-k, s. f.-l).

Elaboración propia.

Algunas de estas respuestas tienen su propio costo. Filtrar en el cliente funciona con el volumen de un restaurante, pero descarga todas las mermas del usuario o del local; con años de historial habría que paginar o filtrar por fecha en el servidor. Además, la regla de lotes permite al operativo disminuir `cantidadDisponible`, pero no comprueba que la disminución coincida con una merma real; un usuario malintencionado podría descontar más de lo registrado. Verificarlo en las reglas requeriría leer la merma dentro de la misma escritura, lo que agrega complejidad y lecturas cobradas.

Por último, varias escrituras se envían sin esperar la confirmación del servidor para aprovechar la cola sin conexión. La consecuencia es que, si las reglas rechazan una de ellas, el usuario no ve el error en ese momento. En el ingreso de lotes y en la configuración de motores ya esperamos la confirmación con un tiempo límite, y queda pendiente hacer lo mismo con el registro de mermas. La regla de ventas simuladas tiene un riesgo parecido al de los lotes: cualquier usuario activo puede crear ventas `SIM-*`, y aunque no puede tocar las reales, sí podría inflar las simuladas y mover la recomendación de producción.

Qué haríamos distinto

Si empezáramos de nuevo, definiríamos las reglas de seguridad junto con el modelo de datos y no después. Varios ajustes de este avance, como la regla de lotes o la consulta de mermas por rol, aparecieron porque las reglas se escribieron pensando en colecciones y no en las consultas reales de cada pantalla. También usaríamos el emulador local de Firebase desde el inicio, para probar las reglas con pruebas automatizadas antes de publicarlas.

Conclusiones

Con este avance, los datos de Scrapp dejaron de vivir en memoria y pasaron a la nube. Firebase Authentication resolvió el inicio de sesión con correo y con Google, la recuperación de

contraseña y la sesión persistente, y lo unimos al rol guardado en Firestore para que la navegación y los permisos dependan del perfil de cada usuario. Una cuenta desactivada pierde el acceso en segundos, aunque tenga la sesión abierta.

Cloud Firestore aloja ahora los productos, lotes, mermas, ventas y la configuración de los motores, con cada registro asociado al `uid` de quien lo creó y con fechas del servidor. El modelo de datos se diseñó a partir de las consultas de la app, con desnormalización donde la falta de `JOIN` lo exigía, y las reglas de seguridad aplican en el servidor los permisos de cada rol. La sincronización en tiempo real, construida con *listeners* y `StateFlow`, permite que lo registrado en cocina aparezca en el panel de gerencia sin recargar, que es el problema que la aplicación busca resolver desde la Etapa 1.

Para la entrega final quedan dos tareas: confirmar las escrituras de merma para mostrar al usuario los errores de reglas y reforzar las reglas de descuento de lotes y de ventas simuladas. Ninguna de las dos obliga a cambiar la arquitectura.

Referencias

Android Developers. (s. f.-a). *About Sign in with Google*. Google. Recuperado el 4 de noviembre de 2026, de https://developer.android.com/identity/sign-in/credential-manager-siwg

Android Developers. (s. f.-b). *StateFlow and SharedFlow*. Google. Recuperado el 4 de noviembre de 2026, de https://developer.android.com/kotlin/flow/stateflow-and-sharedflow

Firebase. (s. f.-a). *Access data offline*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/manage-data/enable-offline

Firebase. (s. f.-b). *Add Firebase to your Android project*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/android/setup

Firebase. (s. f.-c). *Authenticate with Firebase using password-based accounts on Android*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/auth/android/password-auth

Firebase. (s. f.-d). *Authenticate with Google on Android*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/auth/android/google-signin

Firebase. (s. f.-e). *Cloud Firestore data model*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/data-model

Firebase. (s. f.-f). *Default bucket and billing requirements for Cloud Storage for Firebase after September 2024*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/storage/faqs-storage-changes-announced-sept-2024

Firebase. (s. f.-g). *Firebase Authentication*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/auth

Firebase. (s. f.-h). *Get realtime updates with Cloud Firestore*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/query-data/listen

Firebase. (s. f.-i). *Securely query data*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/security/rules-query

Firebase. (s. f.-j). *Transactions and batched writes*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/manage-data/transactions

Firebase. (s. f.-k). *Understand Cloud Firestore billing*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/pricing

Firebase. (s. f.-l). *Writing conditions for Cloud Firestore Security Rules*. Google. Recuperado el 4 de noviembre de 2026, de https://firebase.google.com/docs/firestore/security/rules-conditions

Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9